# 5.1. 제어 및 상태 레지스터 명령어 확장 Zicsr 버전 2.0

한국어 비공식 번역 · RISC-V ISA 명세 20260120

원문 zicsr.html

원문의 절 번호와 표 번호를 유지했다. 명령어 및 레지스터 이름은 원문 표기를 사용하며, 비트 도식은 동일한 필드와 비트 범위를 표로 재현했다.

RISC-V는 각 하트(hart)에 연결된 4,096개의 제어 및 상태 레지스터(Control and Status Register, CSR)를 위한 별도의 주소 공간을 정의한다. 이 장은 이러한 CSR을 대상으로 동작하는 CSR 명령어 전체를 정의한다.

> **주석** CSR은 주로 특권 아키텍처에서 사용되지만, 비특권 코드에서도 카운터와 타이머, 부동소수점 상태 등 여러 용도로 사용된다. 카운터와 타이머는 더 이상 표준 기본 ISA의 필수 구성 요소로 간주되지 않는다. 따라서 이들에 접근하는 데 필요한 CSR 명령어를 「RV32I 기본 정수 명령어 집합 버전 2.1」에서 분리하여 이 장으로 옮겼다.

## 5.1.1. CSR 명령어

모든 CSR 명령어는 하나의 CSR에 대해 원자적인 읽기-수정-쓰기(read-modify-write)를 수행한다. 대상 CSR을 지정하는 번호는 명령어의 비트 31~20에 위치한 12비트 `csr` 필드에 인코딩된다. 즉시값 형식은 `rs1` 필드에 인코딩된 5비트 즉시값을 0 확장하여 사용한다.

| 명령어 | 31:20 | 19:15 | 14:12 | 11:7 | 6:0 |
|---|---|---|---|---|---|
| CSRRW | csr | rs1 | 001 | rd | 1110011 |
| CSRRS | csr | rs1 | 010 | rd | 1110011 |
| CSRRC | csr | rs1 | 011 | rd | 1110011 |
| CSRRWI | csr | uimm[4:0] | 101 | rd | 1110011 |
| CSRRSI | csr | uimm[4:0] | 110 | rd | 1110011 |
| CSRRCI | csr | uimm[4:0] | 111 | rd | 1110011 |

CSRRW(Atomic Read/Write CSR, CSR 원자적 읽기/쓰기) 명령어는 CSR과 정수 레지스터의 값을 원자적으로 교환한다. CSRRW는 CSR의 이전 값을 읽고 XLEN비트로 0 확장한 뒤 정수 레지스터 `rd`에 쓴다. `rs1`의 초기 값은 CSR에 쓴다. `rd=x0`이면 이 명령어는 CSR을 읽지 않아야 하며, CSR 읽기로 발생할 수 있는 어떠한 부수 효과도 일으키지 않아야 한다.

CSRRS(Atomic Read and Set Bits in CSR, CSR 원자적 읽기 및 비트 설정) 명령어는 CSR의 값을 읽고 XLEN비트로 0 확장한 뒤 정수 레지스터 `rd`에 쓴다. 정수 레지스터 `rs1`의 초기 값은 CSR에서 1로 설정할 비트 위치를 지정하는 비트 마스크로 취급된다. `rs1`에서 1인 각 비트는, 대응하는 CSR 비트가 쓰기 가능할 경우 그 비트를 1로 설정한다.

CSRRC(Atomic Read and Clear Bits in CSR, CSR 원자적 읽기 및 비트 지우기) 명령어는 CSR의 값을 읽고 XLEN비트로 0 확장한 뒤 정수 레지스터 `rd`에 쓴다. 정수 레지스터 `rs1`의 초기 값은 CSR에서 0으로 지울 비트 위치를 지정하는 비트 마스크로 취급된다. `rs1`에서 1인 각 비트는, 대응하는 CSR 비트가 쓰기 가능할 경우 그 비트를 0으로 지운다.

> **주석** CSRRS와 CSRRC는 읽기-수정-쓰기를 수행하므로, 실제 내부 값과 다른 값으로 읽히는 비트는 `rs1`의 대응 비트가 1로 설정되지 않았더라도 이 명령어에 의해 변경될 수 있다. 예를 들어 `pmpaddrn[G-1]`의 실제 내부 값은 1이지만 읽을 때는 0으로 나타날 수 있다. 다른 비트를 수정하기 위해 CSRRC 또는 CSRRS를 실행하면, `pmpaddrn[G-1]`에서 0을 읽은 다음 그 값을 다시 쓰므로 실제 내부 값이 0으로 바뀐다.

CSRRS와 CSRRC 모두 `rs1=x0`이면 CSR에 전혀 쓰지 않는다. 따라서 CSR 쓰기로 발생할 수 있는 부수 효과를 일으키지 않으며, 읽기 전용 CSR에 접근할 때 불법 명령어 예외(illegal-instruction exception)도 발생시키지 않는다.

CSRRS와 CSRRC는 `rs1` 및 `rd` 필드에 관계없이 지정된 CSR을 항상 읽으며, 읽기에 따른 부수 효과가 있으면 이를 발생시킨다.

`rs1`이 `x0`이 아닌 레지스터를 지정하고 그 레지스터에 값 0이 들어 있는 경우에는, 관련된 필드별 부수 효과는 발생시키지 않지만 CSR 전체에 쓰는 동작으로 인한 부수 효과는 발생시킨다는 점에 유의해야 한다.

`rs1=x0`인 CSRRW는 대상 CSR에 0을 쓰려고 시도한다.

CSRRWI, CSRRSI, CSRRCI는 각각 CSRRW, CSRRS, CSRRC와 유사하다. 차이점은 정수 레지스터 값 대신 `rs1` 필드에 인코딩된 5비트 부호 없는 즉시값 `uimm[4:0]`을 XLEN비트로 0 확장한 값으로 CSR을 갱신한다는 것이다.

CSRRSI와 CSRRCI에서 `uimm[4:0]` 필드가 0이면 CSR에 쓰지 않으며, CSR 쓰기로 발생할 수 있는 부수 효과나 읽기 전용 CSR 접근에 대한 불법 명령어 예외를 발생시키지 않아야 한다. CSRRWI에서 `rd=x0`이면 CSR을 읽지 않아야 하며, CSR 읽기로 발생할 수 있는 부수 효과도 발생시키지 않아야 한다.

CSRRSI와 CSRRCI는 `rd` 및 `rs1` 필드에 관계없이 CSR을 항상 읽으며, 읽기에 따른 부수 효과가 있으면 이를 발생시킨다.

**표 1. CSR 명령어가 지정된 CSR을 읽거나 쓰는지 결정하는 조건**

레지스터 피연산자

| 명령어 | rd가 x0인가 | rs1이 x0인가 | CSR 읽기 | CSR 쓰기 |
|---|---|---|---|---|
| CSRRW | 예 | — | 아니요 | 예 |
| CSRRW | 아니요 | — | 예 | 예 |
| CSRRS/CSRRC | — | 예 | 예 | 아니요 |
| CSRRS/CSRRC | — | 아니요 | 예 | 예 |

즉시값 피연산자

| 명령어 | rd가 x0인가 | uimm=0인가 | CSR 읽기 | CSR 쓰기 |
|---|---|---|---|---|
| CSRRWI | 예 | — | 아니요 | 예 |
| CSRRWI | 아니요 | — | 예 | 예 |
| CSRRSI/CSRRCI | — | 예 | 예 | 아니요 |
| CSRRSI/CSRRCI | — | 아니요 | 예 | 예 |

표 1은 CSR 명령어의 CSR 읽기 및 쓰기 여부를 요약한다.

CSR 읽기나 쓰기 자체에 따른 부수 효과 외에도, CSR 내부의 개별 필드에 쓰기를 수행할 때 부수 효과가 발생할 수 있다. CSRRW[I] 명령어는 쓰기 대상 CSR 안의 그러한 모든 필드에 대해 부수 효과를 발생시킨다. CSRRS[I]와 CSRRC[I] 명령어는 `rs1` 또는 `uimm` 인자에서 해당 필드에 대응하는 비트 중 적어도 하나가 1인 필드에 대해서만 부수 효과를 발생시킨다.

> **주석** 이 문서 작성 시점에는 필드 쓰기에 부수 효과가 있는 표준 CSR이 없다. 따라서 표준 CSR 접근에 부수 효과가 있는지는 오직 명령어의 연산 코드만으로 판단할 수 있다. 필드 쓰기에 부수 효과가 있는 CSR을 정의하는 것은 권장하지 않는다.

CSR이 특정 값을 갖기 때문에 발생하는 사건이나 결과에 대해, CSR 쓰기로 해당 값이 설정되었다면 그 사건이나 결과를 쓰기의 **간접 효과(indirect effect)**라고 한다. RISC-V ISA는 CSR 쓰기의 간접 효과를 해당 쓰기의 **부수 효과(side effect)**로 간주하지 않는다.

**주석** CSR 접근의 부수 효과를 예로 들면, 특정 CSR을 읽으면 전구가 켜지고, 같은 CSR에 홀수를 쓰면 전구가 꺼지는 경우가 있다. 짝수를 쓰면 아무 효과가 없다고 가정하자. 이 경우 전구가 켜졌는지가 CSR 값만으로 결정되지 않으므로, 읽기와 쓰기 모두 전구의 점등 상태를 제어하는 부수 효과를 갖는다.

홀수를 써서 전구를 끈 다음, 읽어서 전구를 켜고, 다시 같은 홀수를 쓰면 전구가 다시 꺼진다. 따라서 마지막 쓰기에서는 CSR 값의 변화가 전구를 끄는 원인이 아니다. 반면 특정 CSR 값이 홀수일 때마다 전구가 켜지도록 연결되어 있다면, 전구가 켜지고 꺼지는 현상은 CSR 쓰기의 부수 효과가 아니라 그 쓰기의 간접 효과로 간주한다.

보다 구체적으로, 제II권에 정의된 RISC-V 특권 아키텍처에서는 특정 CSR 값의 조합이 트랩을 발생시키도록 규정한다. CSR에 대한 명시적 쓰기가 트랩 조건을 만들었을 때, 그 트랩은 쓰기의 부수 효과가 아니라 간접 효과로 간주한다. 표준 CSR에는 읽기 부수 효과가 없다. 표준 CSR에는 쓰기 부수 효과가 있을 수 있다. 사용자 정의 확장은 읽기 또는 쓰기 접근에 부수 효과가 있는 CSR을 추가할 수 있다.

명령어 완료 수 카운터 `instret`와 같은 일부 CSR은 명령어 실행의 부수 효과로 수정될 수 있다. 이 경우 CSR 접근 명령어가 해당 CSR을 읽으면 그 명령어 실행 전의 값을 읽는다. CSR 접근 명령어가 해당 CSR에 쓰면, 부수 효과에 따른 갱신 대신 명시적 쓰기가 수행된다. 특히 어떤 명령어가 `instret`에 쓴 값은 바로 다음 명령어가 읽는 값이 된다.

CSR을 읽는 어셈블러 의사 명령어 `CSRR rd, csr`는 `CSRRS rd, csr, x0`으로 인코딩된다. CSR에 쓰는 의사 명령어 `CSRW csr, rs1`은 `CSRRW x0, csr, rs1`으로, `CSRWI csr, uimm`은 `CSRRWI x0, csr, uimm`으로 인코딩된다.

이전 값이 필요하지 않을 때 CSR 비트를 설정하거나 지우기 위한 의사 명령어도 정의된다. `CSRS/CSRC csr, rs1` 및 `CSRSI/CSRCI csr, uimm`이 이에 해당한다.

### 5.1.1.1. CSR 접근 순서

각 RISC-V 하트는 일반적으로 암묵적 CSR 접근을 포함한 자신의 CSR 접근이 프로그램 순서대로 수행되는 것으로 관찰한다. 특히 별도 규정이 없으면, CSR 접근은 프로그램 순서상 앞선 명령어 중 그 동작이 CSR 상태를 수정하거나 CSR 상태의 영향을 받는 모든 명령어가 실행된 뒤에 수행되고, 뒤따르는 명령어 중 그 동작이 CSR 상태를 수정하거나 CSR 상태의 영향을 받는 모든 명령어가 실행되기 전에 수행된다.

또한 명시적 CSR 읽기는 해당 명령어 실행 전의 CSR 상태를 반환한다. 명시적 CSR 쓰기는 같은 명령어가 같은 CSR에 수행하는 모든 암묵적 쓰기나 수정을 억제하고 그 결과를 덮어쓴다.

마찬가지로 명시적 CSR 접근에 따른 부수 효과도 일반적으로 프로그램 순서에 맞추어 동기적으로 발생하는 것으로 관찰된다. 별도 규정이 없으면, 바로 다음 명령어에서 그러한 부수 효과의 전체 결과를 관찰할 수 있어야 하며, 앞선 명령어가 프로그램 순서를 벗어나 그 결과를 먼저 관찰해서는 안 된다. 앞에서 구분한 CSR 쓰기의 부수 효과와 간접 효과의 차이에 유의해야 한다.

RVWMO 메모리 일관성 모델(「RVWMO 메모리 일관성 모델」 참조)에서 CSR 접근은 기본적으로 약하게 순서화된다. 따라서 다른 하트나 장치는 CSR 접근을 프로그램 순서와 다른 순서로 관찰할 수 있다.

또한 다음 경우를 제외하면 CSR 접근과 명시적 메모리 접근 사이에는 순서가 보장되지 않는다. CSR 접근이 명시적 메모리 접근을 수행하는 명령어의 실행 동작을 바꾸는 경우, 또는 메모리 모델에 정의된 구문적 의존성이나 이 설명서 제II권의 「메모리 순서 PMA」 절에서 정의한 순서 요구사항에 따라 두 접근의 순서가 정해지는 경우이다.

그 밖의 모든 경우에 순서를 강제하려면 소프트웨어는 관련 접근 사이에 FENCE 명령어를 실행해야 한다. FENCE 명령어의 관점에서 CSR 읽기 접근은 장치 입력(I), CSR 쓰기 접근은 장치 출력(O)으로 분류된다.

**주석** 비공식적으로 표현하면, CSR 공간은 이 설명서 제II권의 「메모리 순서 PMA」 절에 정의된 약하게 순서화된 메모리 매핑 I/O 영역처럼 동작한다. 따라서 다른 모든 접근에 대한 CSR 접근의 순서는, 그러한 영역에 대한 메모리 매핑 I/O 접근의 순서를 제한하는 것과 같은 메커니즘에 의해 제한된다.

이러한 CSR 순서 제약은 장치나 다른 하트가 볼 수 있거나 그 영향을 받는 CSR 접근과 주 메모리 및 메모리 매핑 I/O 접근 사이의 순서를 지원하기 위한 것이다. 예를 들어 `time`, `cycle`, `mcycle` CSR과 대기 중인 인터럽트를 나타내는 `mip`, `sip` 등이 해당한다. 이러한 CSR의 암묵적 읽기, 예를 들어 `mip` 변화로 인터럽트를 수신하는 동작도 장치 입력으로 순서화된다는 점에 유의해야 한다.

`fcsr` 등을 포함한 대부분의 CSR은 다른 하트에 보이지 않는다. 따라서 그 접근은 이 명세를 위반하지 않으면서 전역 메모리 순서에서 FENCE 명령어에 대해 자유롭게 재정렬될 수 있다.

하드웨어 플랫폼은 특정 CSR에 대한 접근을, 이 설명서 제II권의「메모리 순서 PMA」절에서 정의한 강하게 순서화된 접근으로 지정할 수 있다. 강하게 순서화된 CSR 접근에는 약하게 순서화된 CSR 접근 및 메모리 매핑 I/O 영역 접근에 대해 더 강한 순서 제약이 적용된다.

> **주석** 전역 메모리 순서에서 CSR 접근을 재정렬하는 규칙은 아마도 RVWMO 메모리 일관성 모델을 다루는「RVWMO 메모리 일관성 모델」장으로 옮기는 것이 적절할 것이다.

## 출처 및 이용 조건

원저작물 RISC-V International 및 원저자. The RISC-V Instruction Set Manual. 공식 릴리스 20260120. 이 문서는 요청된 웹페이지 본문의 한국어 번역이며 RISC-V International의 공식 번역이 아니다. 웹사이트 탐색 메뉴와 공통 푸터는 제외했다.

© RISC-V International. 원저작물은 Creative Commons Attribution 4.0 International CC BY 4.0으로 제공된다. 한국어 번역과 서식 재구성을 적용했다. 원저작물의 보증 부인 및 라이선스 조건은 아래 링크를 따른다.

CC BY 4.0 라이선스

공식 원문 및 기여자